Software Requirements Specification for the Electronics Store Inventory Management System (SEIMS)

Karthik Peddibhotla
Department of Computer Science & Engineering
PES University
SRN: PES2UG24CS348
karthikpeddibhotla5@gmail.com

Paridhi Bajpai
Department of Computer Science & Engineering
PES University
SRN: PES2UG24CS337
paridhi.bajaj03@gmail.com

Niharika S Murthy
Department of Computer Science & Engineering
PES University
SRN: PES2UG24CS316
niharika.sm0@gmail.com

P. Dhanush
Department of Computer Science & Engineering
PES University
SRN: PES2UG24CS333
penchalpaadudhanush@gmail.com

***Abstract*—This Software Requirements Specification (SRS) defines the functional, non-functional, interface, data, security, and behavioral requirements for the Electronics Store Inventory Management System (SEIMS), developed for the Software Engineering course (UE24CS302) at PES University. The system is a browser-based application for managing electronic component inventory. It provides user authentication, dashboard summaries, component record management, search and filtering, stock classification, availability reporting, and CSV export.**

***Index Terms*—Software Requirements Specification, SEIMS, inventory management, Flask, SQLite, requirements engineering, software testing.**

I. Introduction

A. Purpose

This document specifies the requirements for the Electronics Store Inventory Management System. It serves as a baseline for system development, architectural design, testing, and project evaluation. It describes what the system is expected to do and the quality and security constraints that apply.

B. Scope

SEIMS is a web-based application intended to support electronic component inventory tracking and management. Its capabilities include secure user authentication, dashboard summary analytics, component Create, Read, Update, and Delete (CRUD) operations, name-based search, category and availability filtering, stock classification (Available, Low Stock, and Out of Stock), availability reporting, and CSV export.

C. Definitions and Acronyms

TABLE I
Definitions and Acronyms

Term	Meaning
SRS	Software Requirements Specification
SEIMS	Electronics Store Inventory Management System
SKU	Stock Keeping Unit
CRUD	Create, Read, Update, and Delete operations
Low Stock	Quantity less than or equal to the minimum stock threshold
Flask	Python micro web framework
SQLite	Embedded relational database engine

D. Document Overview

The remainder of this document describes the product context, user class, operating environment, functional and non-functional requirements, external interfaces, data requirements, system models, acceptance criteria, and requirement traceability.

II. Overall Description

A. Product Perspective

SEIMS functions as an independent, browser-based web application. The frontend uses HTML5, CSS3, and Jinja2 templates. It communicates through HTTP requests with a Python Flask server, which interacts with a persistent SQLite database (`database.db`) and returns rendered HTML pages or CSV data.

B. Product Functions

The system is intended to:

- Authenticate store personnel using a username and password.
- Display dashboard counts for total components, total quantity, low-stock items, and out-of-stock items.
- Create, view, edit, and delete component records.
- Search component records by name and filter by category or availability.

- Classify stock as Available, Low Stock, or Out of Stock.
- Generate stock availability reports.
- Export inventory records as downloadable CSV files.

C. User Classes

The principal user class is the Admin/Store Manager. Users are expected to have basic web-navigation skills and familiarity with electronic components such as microcontrollers, sensors, integrated circuits, and passive components.

D. Operating Environment

- Server runtime: Python 3.10 or later.
- Web framework: Flask.
- Database engine: SQLite 3.
- Client: A modern browser such as Chrome, Firefox, Safari, or Edge.
- Local development communication: HTTP over TCP port 5000.

E. Design Constraints, Assumptions, and Dependencies

The system is developed using Python Flask, HTML/CSS, and SQLite. State-changing inventory endpoints require an authenticated session. Database path configuration supports an environment variable with a fallback to `database.db`. The system assumes that an initial administrative account exists and that client devices can connect to the Flask server.

III. Specific Requirements

A. Functional Requirements

Table II lists the functional requirements. Priority indicates the relative importance of each requirement.

B. External Interface Requirements

1) User Interface: The user interface consists of responsive web templates, including `login.html`, `dashboard.html`, `inventory.html`, `add_component.html`, `edit_component.html`, and `report.html`. Styling is provided through the project's CSS stylesheet.

2) Software Interfaces: The backend uses Flask routing and request handlers. Database operations use Python's `sqlite3` module, including row-based access through `sqlite3.Row` where configured.

3) Communication Interfaces: The application uses HTTP/1.1 over TCP port 5000 in the local development environment. HTTPS is required for a production deployment.

C. Non-Functional Requirements

IV. System Models

A. System Context

Fig. 1 illustrates the interaction between the Store Manager, browser, Flask backend, and SQLite database.

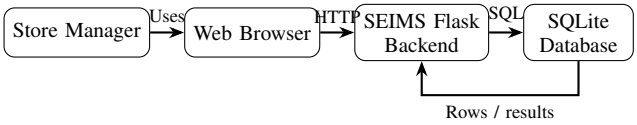


Fig. 1. SEIMS System Context

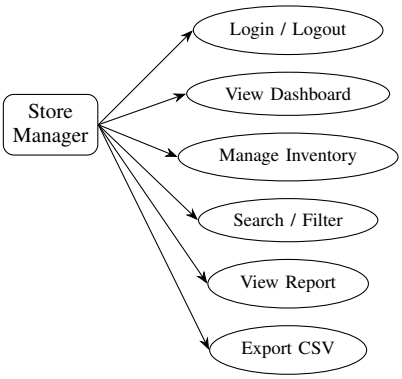


Fig. 2. SEIMS Use-Case Overview

B. Use Cases

The primary actor is the Store Manager. The use cases are:

- UC-01: User Login
- UC-02: View Dashboard
- UC-03: Manage Inventory
- UC-04: Search and Filter Inventory
- UC-05: View Stock Availability Report
- UC-06: Export CSV Data
- UC-07: User Logout

V. Data Requirements

A. Data Entities

The system stores user and component information. The component entity includes `component_id` (integer primary key), `sku` (unique text), `component_name`, `category`, `quantity` (non-negative integer), `price` (non-negative real), and `min_stock_level` (non-negative integer, default 10). The user entity includes `user_id` (integer primary key), `username` (unique text), and `password_hash`.

VI. Acceptance Criteria

The system is considered acceptable when the following criteria are demonstrated:

1) AC-01 (Security): Unauthenticated users are redirected to the login page when accessing protected routes.
2) AC-02 (CRUD): Creating a component updates the stored inventory records.
3) AC-03 (Classification): Stock status is correctly classified as Out of Stock, Low Stock, or Available according to the implemented threshold logic.
4) AC-04 (Export): The downloaded CSV contains the inventory data returned by the export function.
5) AC-05 (Automated Testing): The automated test suite executes successfully.

TABLE II
FUNCTIONAL REQUIREMENTS SPECIFICATION

ID	Name	Description	Priority
FR-01	User Login	Authenticate a user using a username and password.	High
FR-02	User Logout	Clear the active user session when the user logs out.	High
FR-03	Access Control	Restrict unauthenticated access to protected routes.	High
FR-04	Dashboard Summary	Display total, available, low-stock, and out-of-stock counts.	Medium
FR-05	View Inventory	Display component records and their attributes in a table.	High
FR-06	Add Component	Create a component record with name, category, quantity, price, and minimum stock level.	High
FR-07	Edit Component	Modify an existing component record and update the database.	High
FR-08	Delete Component	Delete a selected component using a state-changing request handler.	Medium
FR-09	Search Inventory	Search component records by substring matching on component name.	Medium
FR-10	Category Filter	Filter displayed records by component category.	Medium
FR-11	Status Filter	Filter records by Available, Low Stock, or Out of Stock status.	Medium
FR-12	Clear Filters	Reset search and filter criteria to the default listing.	Low
FR-13	Stock Classification	Calculate stock status using quantity and minimum stock level.	High
FR-14	Availability Report	Generate a grouped report of available, low-stock, and out-of-stock items.	Medium
FR-15	CSV Export	Return inventory data as a downloadable CSV file.	High
FR-16	SKU Uniqueness	Reject insertion or edits that create a duplicate SKU.	High
FR-17	Input Validation	Validate required text fields and non-negative quantities and prices.	High

TABLE III
NON-FUNCTIONAL AND SECURITY REQUIREMENTS

ID	Category	Requirement	Target / Evidence
NFR-01	Usability	Provide understandable forms, stock-status indicators, and feedback messages.	Task completion target: under 30 seconds.
NFR-02	Performance	Provide responsive page loading for the intended inventory workload.	Target: under 2.0 seconds with 1,000 components; measurement required.
NFR-03	Reliability	Preserve committed database records across application restarts.	Verify persistence through restart testing.
NFR-04	Maintainability	Separate templates, styles, and application logic into maintainable units.	Modular project structure.
NFR-05	Portability	Support execution on Windows, Linux, and macOS.	Python 3.10 or later.
SEC-01	Security	Store passwords as cryptographic hashes using the configured Werkzeug password-hashing helper.	No plaintext password storage.
SEC-02	Security	Protect state-changing endpoints using authenticated-session checks.	Unauthenticated edits are denied.
SEC-03	Security	Clear session state when a user logs out.	Session is invalidated.
SEC-04	Security	Use parameterized SQL queries for database operations.	Avoid SQL injection through query parameterization.

TABLE IV
COMPONENT TABLE SCHEMA

Column	Type	Constraint
component_id	INTEGER	Primary key, auto-increment
sku	TEXT	Unique, not null
component_name	TEXT	Not null
category	TEXT	Not null
quantity	INTEGER	Not null, value ≥ 0
price	REAL	Not null, value ≥ 0
min_stock_level	INTEGER	Default 10, value ≥ 0

TABLE V
USER TABLE SCHEMA

Column	Type	Constraint
user_id	INTEGER	Primary key, auto-increment
username	TEXT	Unique, not null
password_hash	TEXT	Not null; stores password hash

VII. REQUIREMENTS TRACEABILITY

VIII. APPENDIX: PROJECT VERIFICATION NOTES

The project repository contains the Flask application, HTML templates, CSS assets, and automated Pytest suites. The project team has reported a local test execution result of 39 passing tests and a successful GitHub Actions workflow run. These are reported project results and should be refreshed against the latest repository state before final submission.

TABLE VI
REQUIREMENTS TRACEABILITY MATRIX

Requirement IDs	Module	Test File
FR-01–FR-03	Authentication	`tests/test_app.py`
FR-04	Dashboard	`tests/test_frontend.py`
FR-05–FR-08	Product CRUD	`tests/test_inventory_backend.py`
FR-09–FR-12	Search and Filtering	`tests/test_frontend.py`
FR-13	Stock Classifier	`tests/test_inventory_backend.py`
FR-14	Availability Reporting	`tests/test_app.py`
FR-15	CSV Export	`tests/test_app.py`
FR-16–FR-17	Validation	`tests/test_inventory_backend.py`

IX. REFERENCES

REFERENCES

[1] IEEE, *IEEE Recommended Practice for Software Requirements Specifications*, IEEE Std 830-1998, 1998.

[2] Python Software Foundation, "Python Documentation." [Online]. Available: https://docs.python.org/3/

[3] Pallets Projects, "Flask Documentation." [Online]. Available: https://flask.palletsprojects.com/

[4] SQLite Development Team, "SQLite Documentation." [Online]. Available: https://www.sqlite.org/docs.html